

DOCUMENTACIÓN TÉCNICA Y FUNCIONAL

CORAZÓN AZTECA

Plataforma web para gimnasios, entrenadores y boxeadores

Proyecto: corazon_jab (Next.js 16 · React 19)

Versión del documento: 1.0

Fecha: Julio 2026

Índice

1. Resumen ejecutivo
2. Objetivo del proyecto
3. Roles de usuario
4. Mapa de páginas por rol
5. Autenticación y control de acceso
6. Integraciones externas
7. Módulo de Análisis Inteligente (IA)
8. Modelo de datos
9. Arquitectura técnica y stack
10. Configuración y variables de entorno
11. Estado del proyecto y trabajo pendiente
12. Glosario

1. Resumen ejecutivo

Corazón Azteca es una plataforma web construida con Next.js orientada a la gestión integral de un gimnasio de boxeo. Conecta tres tipos de usuarios — **administradores**, **entrenadores** y **alumnos** — en un solo sistema, y complementa la gestión deportiva con contenido educativo público (historia del boxeo, leyendas, blog) y con un módulo de inteligencia artificial que asiste al entrenador en la interpretación de datos de sus alumnos.

El proyecto integra tres servicios externos reales y funcionales: **Google Places API** para localizar gimnasios de boxeo cercanos al usuario, **Gemini (Google AI)** para generar insights automáticos y un chat conversacional en el panel del entrenador, y **Supabase** como base de datos y sistema de autenticación de nueva generación, actualmente en proceso de integración.

2. Objetivo del proyecto

Ofrecer una herramienta digital que permita a un gimnasio de boxeo:

- Gestionar el registro deportivo y médico de sus alumnos (peso, nivel, lesiones, títulos de división).
- Dar seguimiento al progreso físico, nutricional y competitivo de cada alumno.
- Facilitar la relación entrenador–alumno, incluyendo horarios, planes de entrenamiento y competencias.
- Difundir contenido educativo y motivacional (historia del boxeo mexicano, leyendas, blog).
- Ayudar al entrenador a tomar decisiones informadas mediante análisis de datos potenciado por inteligencia artificial.
- Permitir a cualquier visitante encontrar gimnasios de boxeo reales cerca de su ubicación.

3. Roles de usuario

Rol	Descripción	Panel de acceso
Administrador	Control total de la plataforma. Aprueba o rechaza artículos del blog enviados por entrenadores/alumnos, y gestiona el directorio general de cuentas (puede eliminar perfiles de alumnos o entrenadores).	/Admin
Entrenador	Administra a sus alumnos (peso, nivel, lesiones, títulos), gestiona horarios de clase, planes de entrenamiento, competencias, y usa el módulo de Análisis Inteligente con IA.	/Entrenador
Alumno (usuario)	Lleva su propio progreso deportivo: entrenamientos, alimentación, historial de peleas, modo campamento antes de una competencia, y puede elegir o cambiar de entrenador.	/Usuario

El acceso a cada panel está protegido por rol: si un usuario sin sesión intenta entrar a una ruta protegida, se le redirige a `/login`; si tiene sesión pero con un rol distinto al requerido por esa sección, se le redirige automáticamente a su propio panel.

4. Mapa de páginas por rol

4.1 Zona pública (sin necesidad de iniciar sesión)

Ruta	Contenido
/	Página de inicio: presentación de la marca, accesos a iniciar sesión / crear cuenta.
/historia	Línea de tiempo de la historia del boxeo en México, desde sus inicios hasta la actualidad.
/leyendas	Perfiles de leyendas del boxeo mexicano e internacional (récord, títulos, época).
/blog, /blog/todos, /blog/escribir	Blog de artículos (nutrición, técnica, psicología, noticias) y formulario para que entrenadores/alumnos envíen contenido a moderación.
/gimnasios	Buscador de gimnasios de boxeo cercanos a la ubicación del usuario, con datos reales de Google Maps.
/entrenadores, /entrenadores/[id]	Directorio público de entrenadores y su perfil individual (especialidad, trayectoria, logros, galería).
/login, /registro/*, /forgot-password	Inicio de sesión y registro (alumno, entrenador, admin), recuperación de contraseña.
/privacidad, /terminos	Páginas legales.

4.2 Panel de Alumno (/Usuario/*)

Ruta	Contenido
/Usuario	Dashboard resumen del alumno.
/Usuario/Entrenamientos	Registro de sesiones de entrenamiento (tipo, duración, intensidad, calificación).
/Usuario/Alimentacion	Plan de alimentación y seguimiento de macronutrientes.
/Usuario/Progreso	Evolución de habilidades técnicas (velocidad, potencia, resistencia, técnica, defensa, ring IQ).
/Usuario/Historial	Récord de peleas disputadas (victorias, derrotas, empates, método).
/Usuario/Contrincante	Próximas peleas programadas y datos del contrincante.
/Usuario/Campamento	"Modo Campamento": preparación intensiva previa a una pelea, con control de peso.
/Usuario/Perfil	Datos personales y deportivos; permite elegir/cambiar de entrenador (directorio, manual o independiente).
/Usuario/mis-articulos	Publicaciones propias enviadas al blog, con su estado de moderación.

4.3 Panel de Entrenador (/Entrenador/*)

Ruta	Contenido
/Entrenador	Dashboard del entrenador.
/Entrenador/Alumnos , /Entrenador/Alumnos/[id]	Listado y detalle de cada alumno: categoría, nivel, salud, plan, evaluaciones, objetivos e historial.
/Entrenador/Analisis	Análisis Inteligente: insights generados por IA y chat conversacional (ver sección 7).
/Entrenador/Horarios	Horarios de clase impartidos por el entrenador.
/Entrenador/Gimnasios	Gimnasios asociados al entrenador.
/Entrenador/planes	Biblioteca de plantillas de plan de entrenamiento.
/Entrenador/competencias	Peleas programadas y torneos de sus alumnos.
/Entrenador/Perfil	Perfil público del entrenador (visible en el directorio).
/Entrenador/mis-articulos	Publicaciones propias enviadas al blog.

4.4 Panel de Administrador (/Admin/*)

Ruta	Contenido
/Admin	Dashboard administrativo.
/Admin/Articulos	Moderación de publicaciones del blog: aprobar o rechazar (con motivo) artículos y logros enviados.
/Admin/Directorio	Gestión global de cuentas: alumnos y entrenadores registrados, con opción de eliminar perfiles.

5. Autenticación y control de acceso

El estado actual de autenticación funciona como un **prototipo funcional basado en** `localStorage`, sin backend real de credenciales. Esto es intencional para la fase de desarrollo actual, pero tiene una limitación de seguridad importante que se detalla más abajo.

5.1 Cómo funciona hoy

- `app/lib/authStorage.ts` guarda las cuentas (correo, contraseña, nombre, rol) en `localStorage` del navegador, sembradas con 15 cuentas de prueba (5 por rol).
- `app/lib/sessionStorage.ts` guarda la sesión activa del usuario logueado.
- `app/components/RequireRole/RequireRole.tsx` protege cada panel: sin sesión → redirige a `/login`; con sesión de otro rol → redirige a su panel correspondiente.
- `app/components/DashboardLayout/DashboardLayout.tsx` provee el sidebar compartido entre los paneles de Usuario y Entrenador (Admin usa su propio layout).

Limitación de seguridad conocida: las contraseñas se guardan en texto plano en `localStorage`, sin cifrado ni backend de verificación. Este patrón es válido únicamente para demostración/desarrollo y **no debe usarse en producción**.

5.2 Migración en curso hacia Supabase Auth

El proyecto ya tiene el cliente de Supabase configurado y funcional (`app/lib/supabase/client.ts`, `server.ts` y un middleware `proxy.ts` que refresca la sesión en cada petición), pero **el flujo de login/registro real todavía no está conectado a Supabase Auth** — sigue usando `localStorage`. La conexión completa es el siguiente paso de la migración, documentado en el spec `.kiro/specs/supabase-database-schema/design.md`, que define cómo cada cuenta de Supabase Auth generará automáticamente una fila en la tabla `accounts` mediante un trigger de base de datos.

6. Integraciones externas

Todas las integraciones corren del lado del servidor (Route Handlers de Next.js); ninguna API key se expone al navegador del cliente.

6.1 Google Places API — Buscador de gimnasios

Aspecto	Detalle
Archivo	<code>app/api/gimnasios/route.ts</code> , <code>app/api/gimnasios/photo/route.ts</code>
Variable de entorno	<code>GOOGLE_MAPS_API_KEY</code>
Servicio de Google	Places API (New) — Text Search
Funcionamiento	Recibe la ubicación (lat/lng) del usuario, busca con la consulta "gimnasio de boxeo" sesgada a esa ubicación, y filtra tipos de lugar no relevantes (tiendas de artículos deportivos, etc.). Las fotos se sirven mediante un proxy propio para no exponer la key.
Página que lo consume	<code>/gimnasios</code>

6.2 Gemini (Google AI) — Análisis Inteligente y Chat

Aspecto	Detalle
Archivos	<code>app/api/analisis-ia/route.ts</code> (insights), <code>app/api/analisis-ia/chat/route.ts</code> (chat)
Variable de entorno	<code>GEMINI_API_KEY</code>
Modelo	<code>gemini-3.6-flash</code>
Funcionamiento	Ver sección 7 (módulo dedicado).

6.3 Supabase — Base de datos y autenticación (en integración)

Aspecto	Detalle
Archivos	<code>app/lib/supabase/client.ts</code> , <code>server.ts</code> , <code>proxy.ts</code> (middleware de refresco de sesión)
Variables de entorno	<code>NEXT_PUBLIC_SUPABASE_URL</code> , <code>NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY</code>
Estado	SDK conectado y funcional; el flujo de negocio (login, registro, lectura/escritura de datos) todavía usa <code>localStorage</code> (ver sección 5.2 y 8).

7. Módulo de Análisis Inteligente (IA)

Disponible en `/Entrenador/Analisis`. Es la funcionalidad más distintiva de la plataforma: usa el modelo Gemini de Google para analizar los datos del grupo de alumnos de un entrenador y generar recomendaciones automáticas.

7.1 Insights automáticos

- Al cargar la página, se envían al servidor los datos de los alumnos del entrenador (categoría, nivel, adherencia al plan, estado de lesión/alta médica, si es campeón de división).
- El servidor (`app/api/analisis-ia/route.ts`) construye un prompt en español y llama a Gemini forzando una respuesta JSON estructurada (`responseSchema`), evitando así tener que interpretar texto libre.
- Gemini devuelve entre 2 y 5 insights clasificados como **Riesgo**, **Oportunidad** o **Predicción**, cada uno con alumno, título, descripción y una acción sugerida.
- Si la API falla o la key no está configurada, la página muestra insights de ejemplo (fallback) sin romperse.

7.2 Chat con la IA

- Un botón flotante ("💬 Preguntar a la IA") abre un panel de chat conversacional.
- Al hacer clic en la acción de un insight (ej. "Programar descanso"), se abre el chat automáticamente con una pregunta pre-cargada sobre ese alumno específico.
- El entrenador puede seguir conversando libremente; cada mensaje se envía a `app/api/analisis-ia/chat/route.ts`, que mantiene el contexto de los alumnos mediante una instrucción de sistema (`systemInstruction`) y el historial de la conversación.

Las secciones de "Tendencias" y "Emparejamiento de Sparring" que aparecen en la misma página son actualmente datos de ejemplo estáticos, no calculados por la IA ni por lógica real — quedan identificados como trabajo pendiente en la sección 11.

8. Modelo de datos

8.1 Estado actual: localStorage (prototipo)

Toda la persistencia funcional hoy vive en el navegador, en archivos bajo `app/lib/`:

Archivo	Qué guarda
<code>authStorage.ts</code>	Cuentas de usuario (correo, contraseña, nombre, rol)
<code>sesionStorage.ts</code>	Sesión activa y peleas próximas
<code>alumnoStorage.ts</code>	Datos deportivos del alumno y su relación con el entrenador
<code>entrenadorStorage.ts</code>	Perfil público del entrenador, galería, redes sociales
<code>blogStorage.ts</code>	Artículos y logros del blog, con flujo de moderación
<code>contenidoStorage.ts</code>	Contenido editorial (imágenes de historia/leyendas)

Los datos en `localStorage` son locales al navegador de cada persona: no se comparten entre dispositivos ni usuarios reales, y se pierden si se limpia la caché del navegador. Es un modelo válido solo para demostración.

8.2 Modelo objetivo: Supabase (diseñado, en migración)

El esquema de base de datos definitivo está completamente diseñado en `.kiro/specs/supabase-database-schema/design.md`, con más de 20 tablas en PostgreSQL y seguridad a nivel de fila (Row Level Security). Las tablas principales son:

Tabla	Propósito
<code>accounts</code>	Identidad y rol de cada usuario, vinculada 1:1 con Supabase Auth
<code>perfiles_deportivos</code>	Datos deportivos del alumno (nivel, peso, relación con entrenador)
<code>perfiles_publicos_entrenador</code>	Perfil público del entrenador (especialidad, trayectoria)
<code>fichas_salud</code>	Estado de lesión y alta médica del alumno
<code>titulos_division</code>	Si el alumno es campeón de su categoría de peso
<code>publicaciones</code>	Artículos y logros del blog, con moderación
<code>gimnasios</code> , <code>horarios_clase</code>	Gimnasios asociados y horarios de clase
<code>registros_pelea</code> , <code>peleas_programadas</code> , <code>torneos</code>	Historial y agenda de competencias
<code>evaluaciones_habilidades</code> , <code>objetivos_alumno</code> , <code>registros_entrenamiento</code>	Seguimiento de progreso técnico y físico
<code>planes_alimentacion</code> , <code>registros_macros</code>	Nutrición y macronutrientes
<code>modos_campamento</code> , <code>controles_peso</code>	Preparación intensiva pre-pelea

Cada tabla tiene políticas de seguridad (RLS) que garantizan, por ejemplo, que un alumno solo pueda ver y modificar sus propios datos, que un entrenador solo acceda a los alumnos que lo tienen asignado desde el directorio, y que un administrador aprobado tenga acceso total. Actualmente solo 2 de las migraciones SQL de este diseño están aplicadas (extensiones e inicialización de `accounts`); el resto del esquema está diseñado pero pendiente de implementación (ver sección 11).

9. Arquitectura técnica y stack

Componente	Tecnología / Versión
Framework	Next.js 16.2.11 (App Router)
Librería UI	React 19.2.4 / React DOM 19.2.4
Lenguaje	TypeScript 5
Estilos	Tailwind CSS 4 + CSS Modules (design tokens centralizados en <code>app/styles/tokens.css</code>)
Base de datos / Auth (en integración)	Supabase (<code>@supabase/supabase-js</code> , <code>@supabase/ssr</code>)
Iconografía	lucide-react
Pruebas	Vitest 3.2.4, Testing Library (React), fast-check (pruebas basadas en propiedades)
APIs externas	Google Places API (New), Gemini API (Google AI)

La aplicación sigue el patrón de *Route Handlers* de Next.js para toda comunicación con servicios externos (`app/api/*`), de modo que ninguna credencial sensible llega al navegador del cliente. Los paneles de Usuario y Entrenador comparten un layout de sidebar común (`DashboardLayout`), mientras que Admin usa una navegación propia.

10. Configuración y variables de entorno

El archivo `.env.local` (no versionado en git) debe contener las siguientes variables para que la aplicación funcione con todas sus integraciones activas:

```
# Google Places API – buscador de gimnasios
GOOGLE_MAPS_API_KEY=

# Supabase – base de datos y autenticación
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=

# Gemini (Google AI) – Análisis Inteligente
GEMINI_API_KEY=
```

Variable	Dónde se obtiene
<code>GOOGLE_MAPS_API_KEY</code>	Google Cloud Console, con "Places API (New)" habilitada
<code>NEXT_PUBLIC_SUPABASE_URL</code> / <code>...PUBLISHABLE_KEY</code>	Panel de Supabase → Project Settings → API
<code>GEMINI_API_KEY</code>	Google AI Studio (aistudio.google.com/apikey)

Sin estas variables, cada integración cae en un modo de respaldo controlado (datos de ejemplo o mensaje de error visible), sin que la aplicación se rompa por completo — excepto la conexión a Supabase, que sí es requerida por el middleware activo en todas las rutas.

11. Estado del proyecto y trabajo pendiente

11.1 Completado y funcional

- Navegación pública completa (historia, leyendas, blog, gimnasios, directorio de entrenadores).
- Paneles de Usuario, Entrenador y Admin con sus secciones principales.
- Sistema de roles y protección de rutas.
- Buscador de gimnasios con datos reales de Google Places.
- Módulo de Análisis Inteligente con insights y chat reales generados por Gemini.
- Cliente de Supabase configurado y funcional a nivel de infraestructura.

11.2 Pendiente

- **Migrar autenticación real a Supabase Auth**, sustituyendo las contraseñas en texto plano de `localStorage`.
- Implementar las migraciones SQL restantes del esquema diseñado en `.kiro/specs/supabase-database-schema/` (actualmente solo 2 de más de 20 tablas están aplicadas).
- Migrar los datos de `app/lib/*.ts` (localStorage) a las tablas de Supabase correspondientes.
- Conectar Supabase Storage para imágenes de perfil, galería y contenido editorial (hoy se guardan como base64 embebido).
- Reemplazar los datos de ejemplo de "Tendencias" y "Emparejamiento de Sparring" en Análisis Inteligente por cálculos reales sobre datos de Supabase.

12. Glosario

Término	Definición
RLS (Row Level Security)	Mecanismo de PostgreSQL/Supabase que restringe qué filas de una tabla puede ver o modificar cada usuario, según reglas definidas a nivel de base de datos.
Route Handler	Archivo <code>route.ts</code> dentro de <code>app/api/</code> en Next.js que expone un endpoint de servidor (equivalente a una API REST).
responseSchema	Parámetro de la API de Gemini que fuerza al modelo a responder en un formato JSON con una estructura específica.
Origen de entrenador	Cómo quedó asociado un alumno a su entrenador: desde el directorio público, agregado manualmente, o de forma independiente (sin entrenador).
Modo Campamento	Sección del panel de alumno para la preparación intensiva previa a una pelea programada, con control de peso.